Féidearthachtaí as Cuimse
Infinite Possibilities

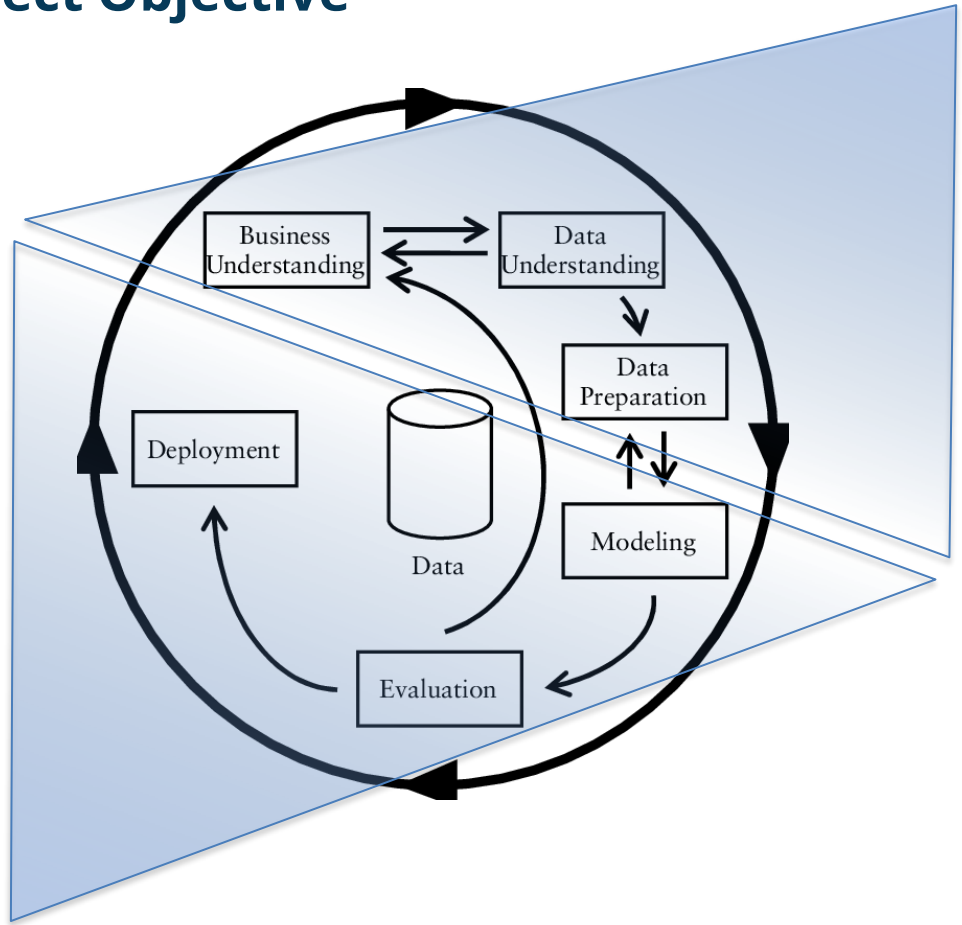
AI: Theory to Practice

Slide Deck

Group 6: Max Sri-Uttha X00213723
Odhran Chamberlain X00217322
Oleh Kravets X00219607

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Project Objective



CA1

In this assignment, we will work in a group of 3 to:

- Identify a publicly available dataset (≥ 1,000 rows, ≥ 8 columns).
- Define a clear supervised machine learning problem (classification or regression).
- Conduct an exploratory data analysis (EDA) (data understanding) and data preparation process to make the dataset model-ready.

CA2

Business Understanding

Credit Risk

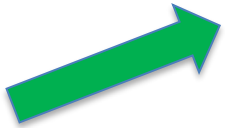


Customers



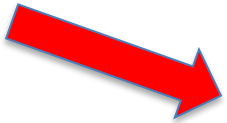


Bank





Credible customers



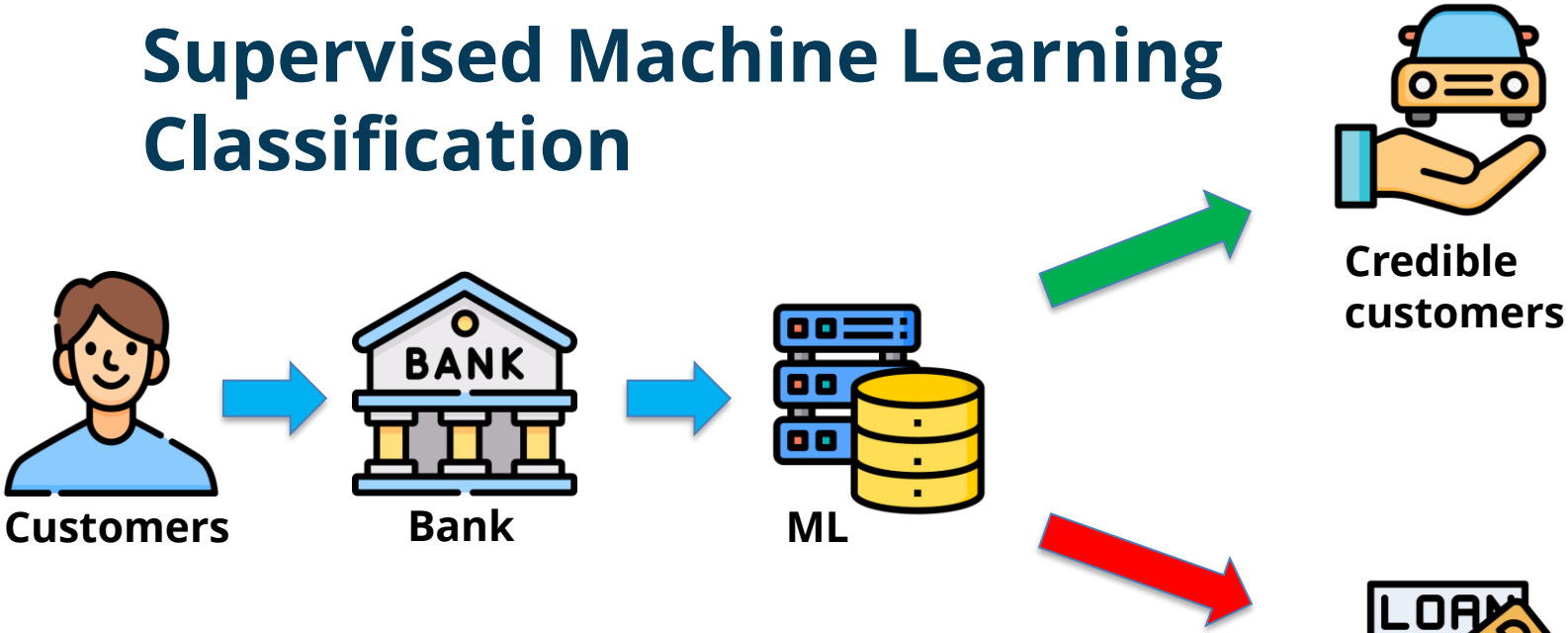


Defaulters

A car-loan is one of many services offered by creditors or financial institutions. Creditors are saddled with the responsibility of credit risk management for any loan facilities offered. To do this, they employ techniques and models to assess the credibility of customers or car loan applicants. The sole aim of this is to distinguish customers into **two categories**: credible customers and defaulters, which means those who are more likely to pay their financial car loan obligations and otherwise, respectively. This will help inform decisions when approving car loans to customers and also help maintain the sustainability of the lender's institution.

Problem Description

Supervised Machine Learning Classification



Data analysis methods such as machine learning are ideal for these purposes. Given the large volume of accumulated real data with known results, we can use **supervised machine learning**. Since the predicted outcomes need to be limited to two customer categories, we can use **classification** instead of **regression**. Numerous datasets with real data for this problem are publicly available. We will use the NBFI Vehicle Loan Repayment Dataset:

https://huggingface.co/datasets/mstz/nbfi

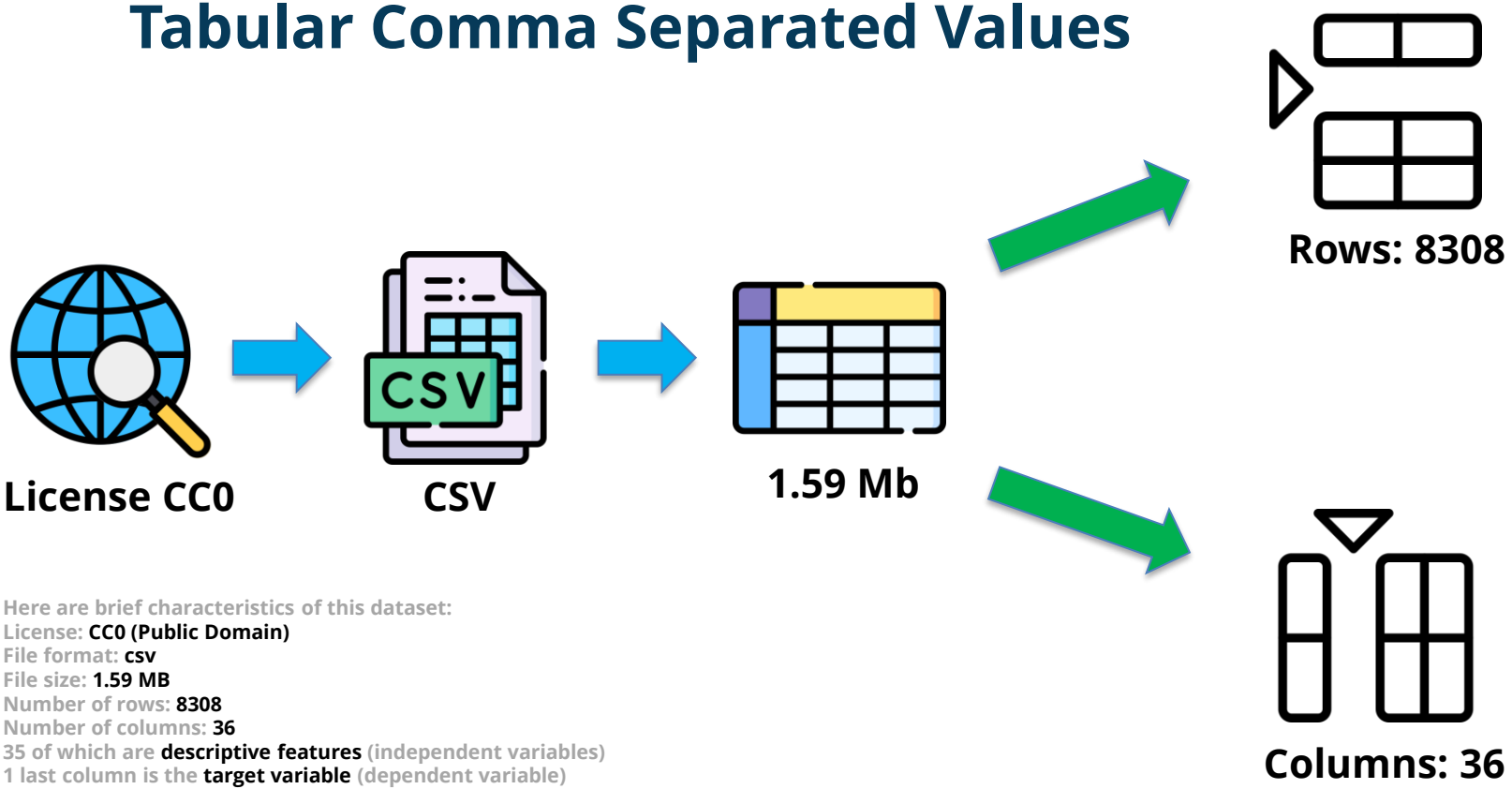
Dataset Source

https://huggingface.co/datasets/mstz/nbfi



Dataset Description

Tabular Comma Separated Values



Here are brief characteristics of this dataset:
License: **CC0 (Public Domain)**
File format: **csv**
File size: **1.59 MB**
Number of rows: **8308**
Number of columns: **36**
35 of which are **descriptive features** (independent variables)
1 last column is the **target variable** (dependent variable)

Data Understanding (EDA)



19 Numeric Columns

In this section we analyze the raw dataset (**df_raw**) before any cleaning. We focus on descriptive statistics, distributions, missingness, outliers, and relationships between variables.



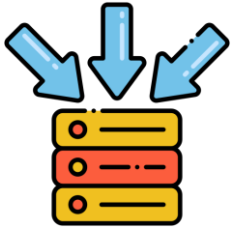
17 Categorical Columns

We generated descriptive stats for all numeric columns in df_raw. For each numeric column we calculated: **count, missing, unique, means, medians, modes, cardinalities, standard deviation, percentiles, min, max, range, IQR, etc.**



Raw Dataset

For each categorical column we report: **number of unique categories, missing count and % missing, most common category (mode), how often that mode appears, what % of the rows that is.**



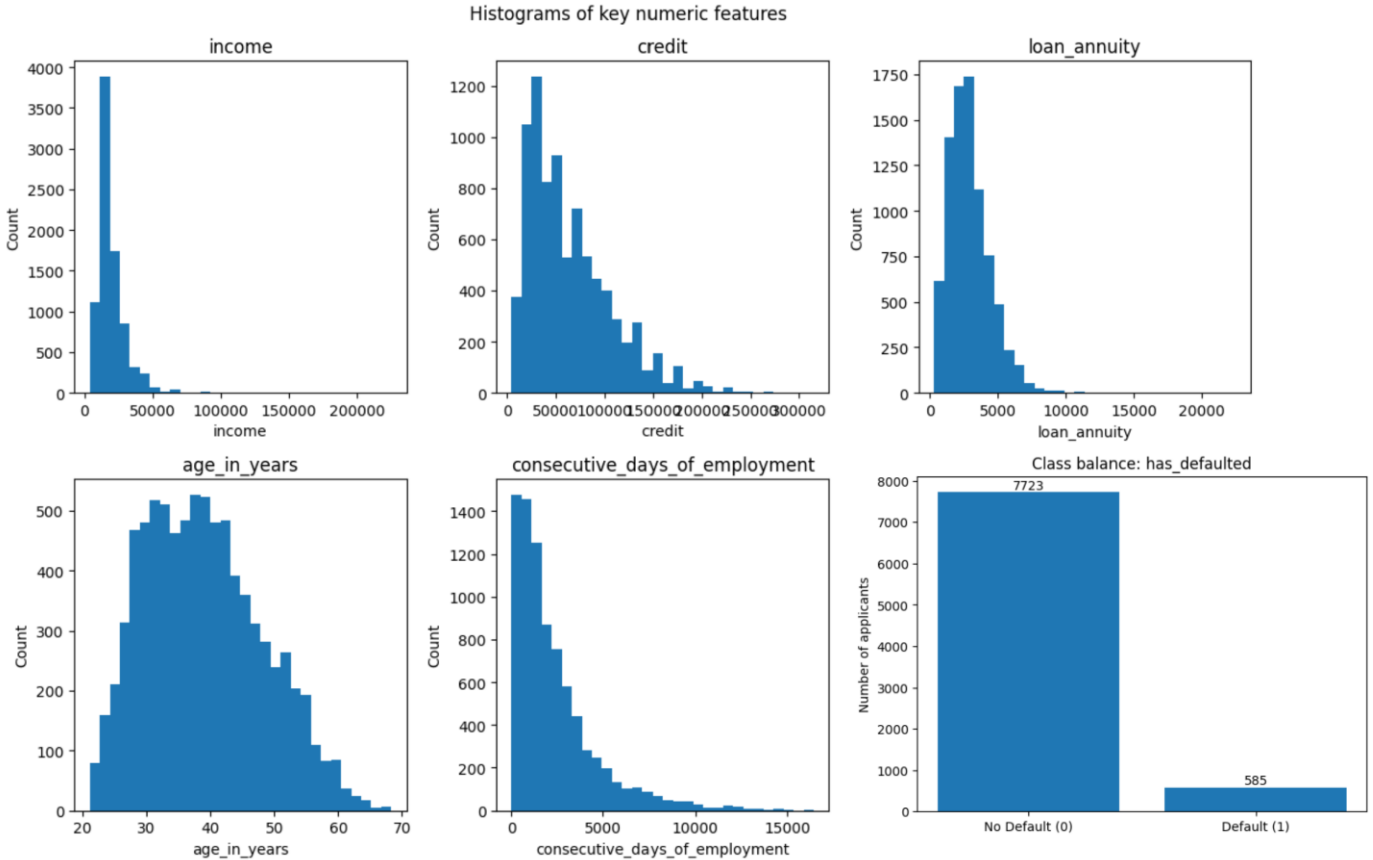
35 Descriptive Features

We also flag data quality issues that will need to be fixed in Data Preparation. During inspection we also noticed some columns that were technically numeric but are really codes or categories (e.g. weekday codes). We handle that later.



1 Target Variable

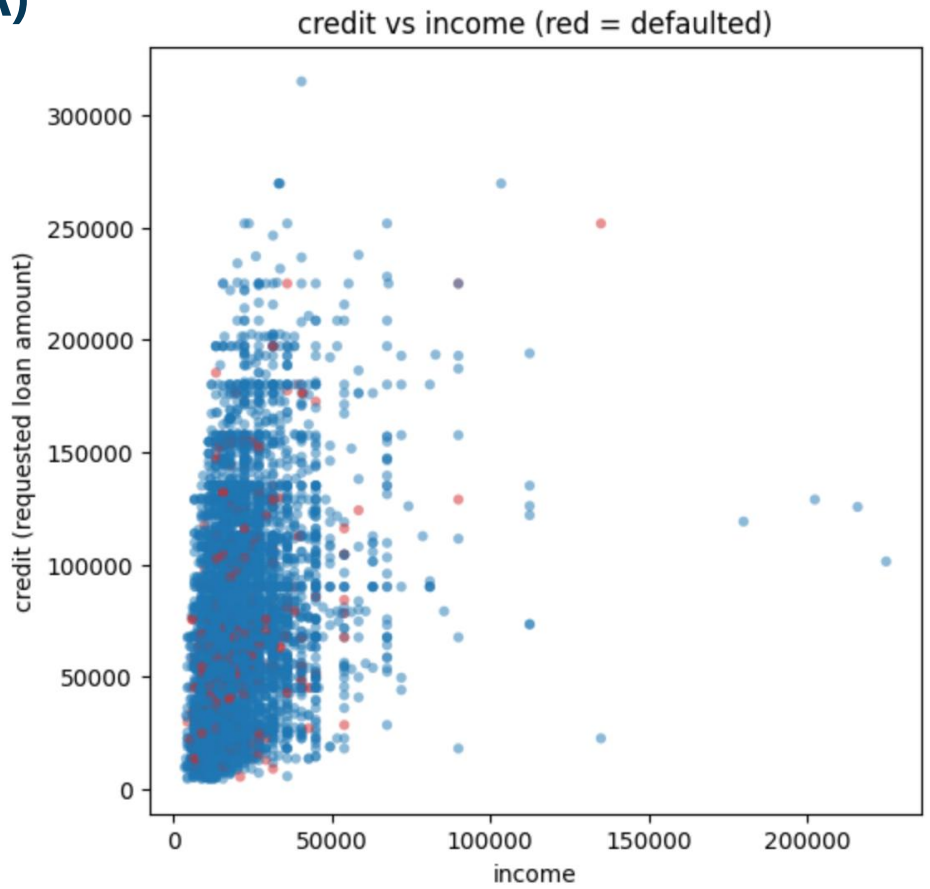
Data Understanding (EDA)



- income is strongly **right-skewed**. Median is reasonable, max is extremely high (200K+). The mean > median, so the tail is pulling the average up.
- credit (requested loan amount) ranges from small to huge (300K+). This matters a lot commercially because the lender's exposure lives here.
- loan_annuity (repayment per period) behaves the same way: it has a long right tail.
- age_in_years looks normal.
- consecutive_days_of_employment shows that some people have extremely long uninterrupted employment histories. Statistically they get flagged as **"outliers"**, but that's stable employment, not necessarily dirty data.
- only ~7% of rows are defaults. Any future model will have to handle class imbalance, because predicting "always 0" would look accurate but be meaningless.

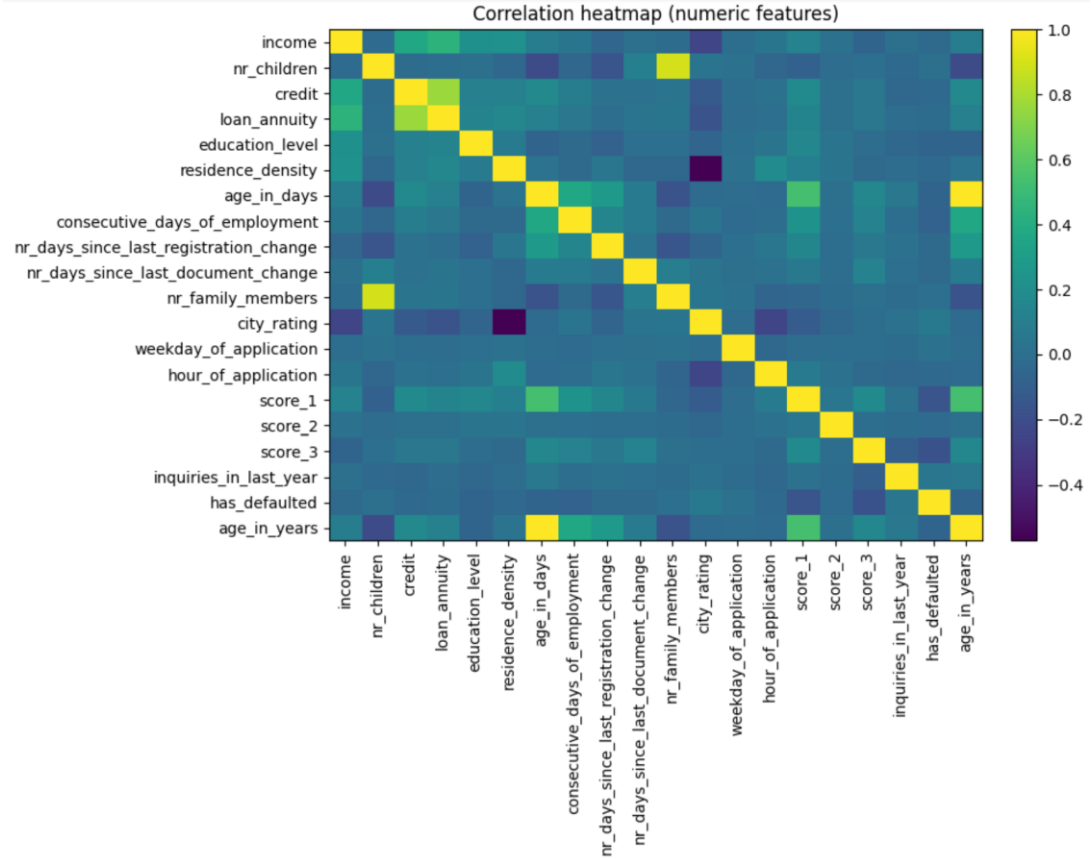
Data Understanding (EDA)

- Higher income tends to come with higher requested credit. That's expected.
- Red points (defaults) are not just clustered in the low-income / low-loan corner. They appear across the range. That tells us we might need affordability-type ratios (like credit / income or loan_annuity / income) instead of just raw values.



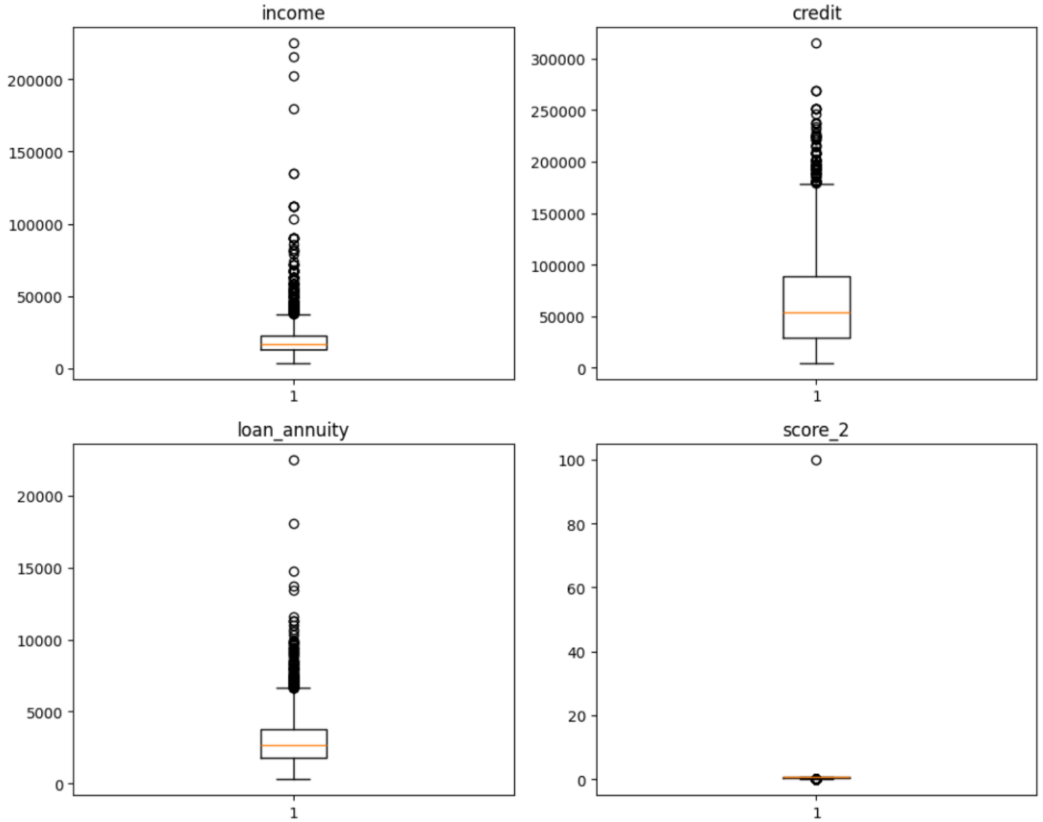
Data Understanding (EDA)

- nr_children and nr_family_members have extremely high correlation (about 0.9). They're basically the same household-size signal.
- credit and loan_annuity also correlate strongly (~0.76), which is natural: bigger loan → higher repayment.
- city_rating and residence_density have extremely low correlation (about 0.1). It is normal.



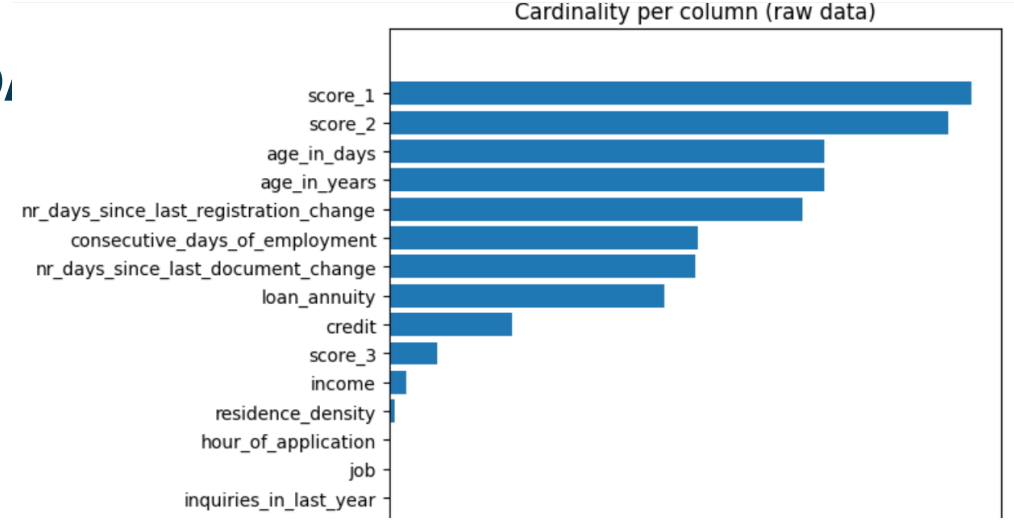
Data Understanding (EDA)

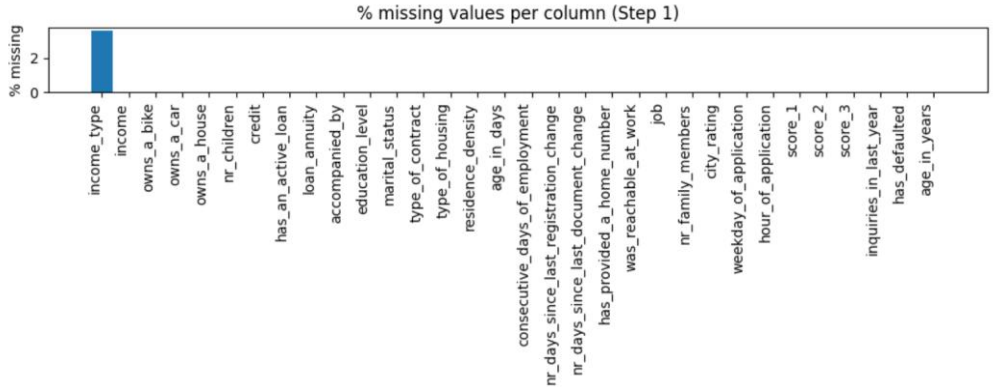
- we see extremely large values for income, credit, and loan_annuity. These are not necessarily “bad data.” They might just be high-exposure
- score_2 is weird: almost all rows are in [0,1], but one row has 100. That one row is definitely off-scale.



Data Understanding (EDA

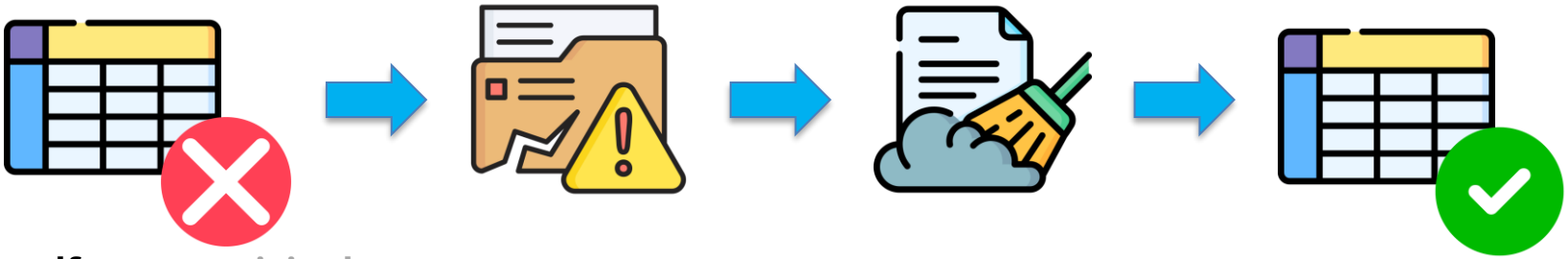
- column with very high cardinality are not useful to the model and can be dropped.
- only income_type has missing values (~3.4%).
- Instead of dropping those rows, we keep them and fill income_type with “Unknown”.
- We want “Unknown” to survive as a separate category so the model can learn from it.





Data Preparation

In this section we take the raw data and make it “model-ready.” We do this in explicit, documented steps. In the notebook, we save intermediate versions so that each step is traceable:



df_raw → original
df_step1 → after basic cleaning
df_step2 → after handling missing values
df_step3 → after handling outliers/extremes
df_step4 → after reducing redundancy
df_step5 → after encoding and feature engineering
X_ready, y → final ABT for modelling

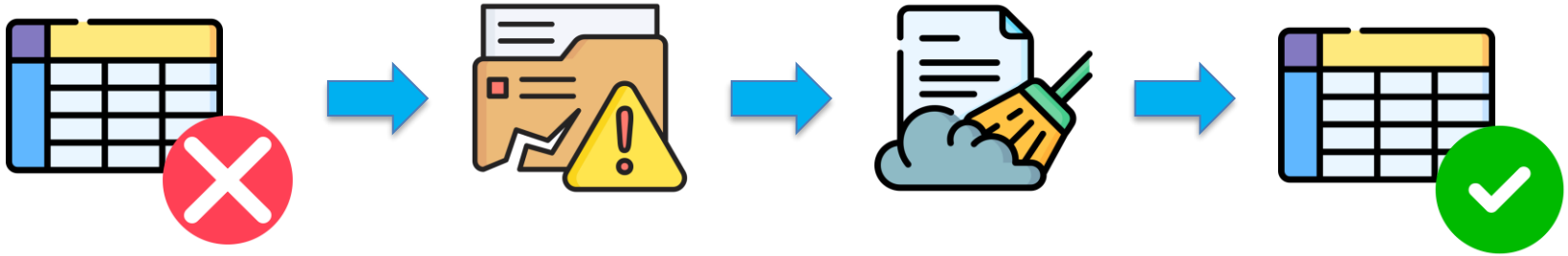
Data Preparation

Duplicates

We found 437 exact duplicate rows in df_raw. We removed them.

- Before removing duplicates: 8,308 rows
- After removing duplicates: 7,871 rows

This prevents the same customer/application from being counted multiple times.



Unimportant columns / constant columns

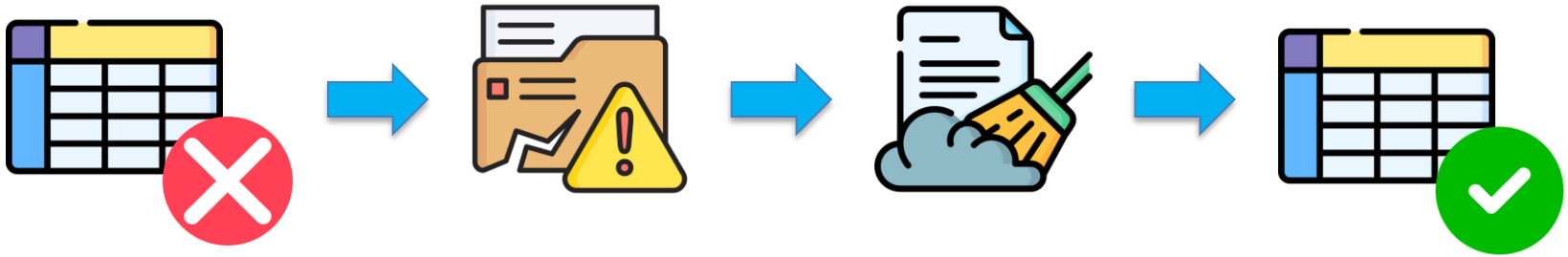
Some columns have the same value for everyone:

- is_male
- has_provided_a_mobile_number
- same_residence_and_home
- same_work_and_home
- nr_defaults_in_social_circle

A feature with zero variance can't help a model distinguish between default and no default. We drop them.

Data Preparation

Incorrect data types



We fixed types for certain columns:
- hour_of_application: cast to int (0–23).
- education_level: keep as integer since it's ordered.
- city_rating: keep as integer since it's ordered.
- weekday_of_application: treat as category, not a numeric quantity.

After these changes we call the dataset df_step1.

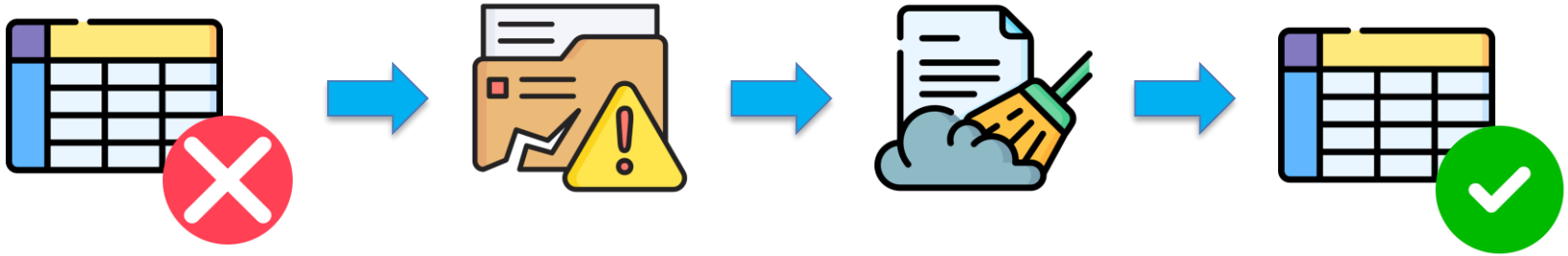
Data Preparation

Handle missing values

Only one feature mattered: income_type (about 3.4% missing). We then checked the default rate in:

- rows where income_type was missing,
- rows where it was provided.

The missing income_type rows showed slightly higher default risk.
That means “did not provide income type” is itself a meaningful behaviour.



Decision:

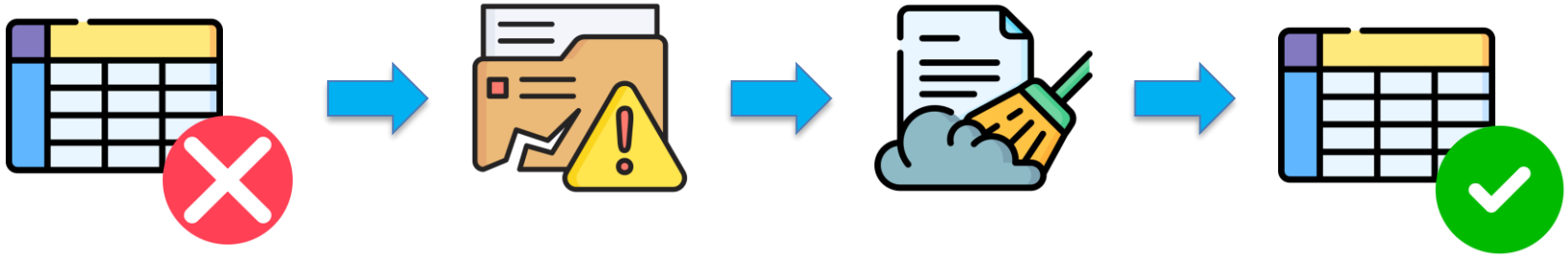
- We do not drop those rows.
- We fill missing income_type with the label "Unknown".
- We keep "Unknown" as its own category going forward so the model can learn that pattern.

After we perform that imputation, we call the dataset df_step2.

Data Preparation

Handle outliers

We do not just delete “outliers.” Instead, we transform them so that:
the model is not dominated by extreme numeric values,
high-risk, high-exposure rows remain in the dataset.



We apply two main strategies:

Cap + log transform on money features

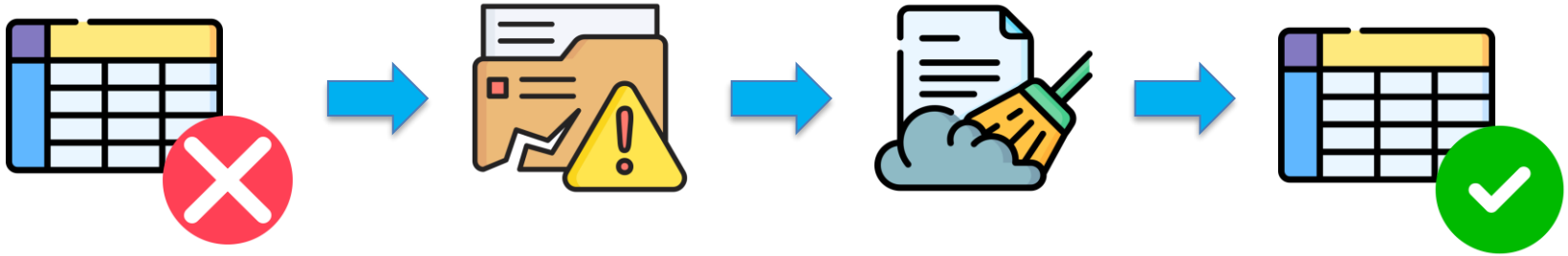
Fix broken scaling in other columns

After applying these transformations and creating these engineered columns we call the dataset df_step3.

Data Preparation

Check for multicollinearity

Some pairs of features are almost the same information. The big one:
nr_children vs nr_family_members have correlation around 0.9.
If we feed both into a model that tries to assign individual importance to each column, that high redundancy can cause unstable behaviour. We don't need two columns saying the same thing.



Action:

Drop nr_children, Keep nr_family_members.

We also saw credit and loan_annuity are highly correlated (~0.76). That's expected (larger loans mean larger payments). We keep both for now, because both contain real business meaning (requested principal vs periodic obligation). If we build linear-style models later, we can regularise or build ratios like loan_annuity / income to express affordability directly.

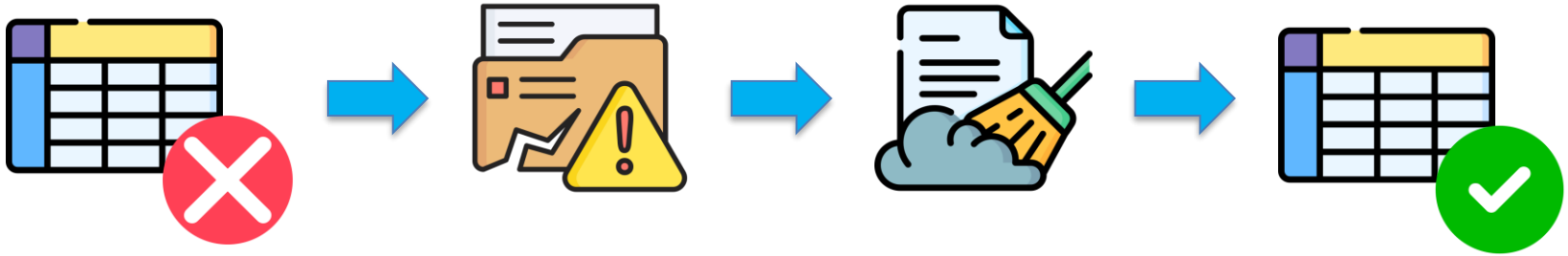
After removing nr_children, we call the dataset df_step4.

Data Preparation

Categorical encoding

Now we convert everything into numeric features in a controlled way and finish the dataset. We split features into six groups and encode each group appropriately.

Binary flags - convert to 0/1 integers.



Nominal categoricals - one-hot encode. We keep "Unknown" (from the imputed income_type) as its own dummy column. We don’t hide it or merge it. That way the model can learn that not declaring income type might be a red flag. For linear models, we can drop one dummy per group to avoid perfect collinearity; for tree-based models we can keep all.

Ordinal categoricals - keep them as integers, because the order matters. We do not one-hot them.

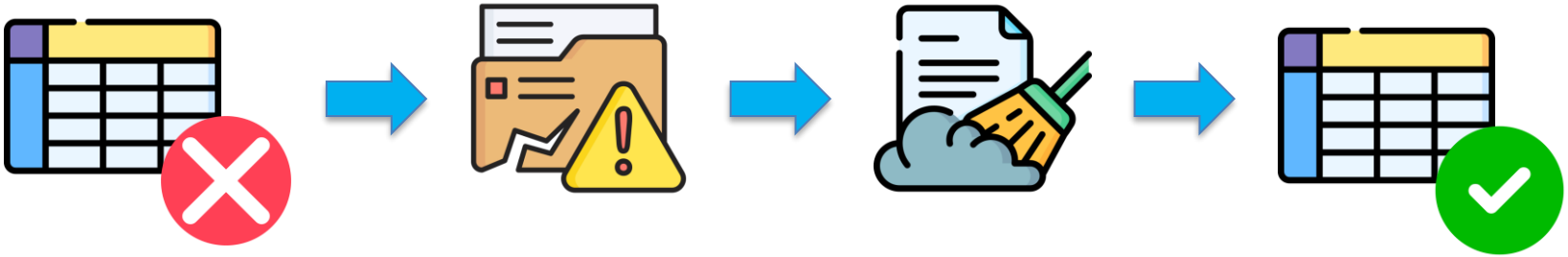
Weekday - weekday_of_application treat as nominal (one-hot).

Hour - this is cyclical (23:00 is “close” to 00:00). We encode it as two numeric features hour_sin and hour_cos. This preserves circular structure across midnight.

Engineered numeric features

Data Preparation

Final ABT



At this point, we assemble the final version for modelling:

X_ready: Final ABT (Analytical Base Table)

y: The target has_defaulted, extracted as a separate vector.

Scaling data

Some models are sensitive to raw feature ranges. For example, credit_log and employment_days_log live on different numeric scales even after engineering. Distance-based models (like k-NN) and certain neural-style models tend to behave better if everything is brought to a common [0,1] range.

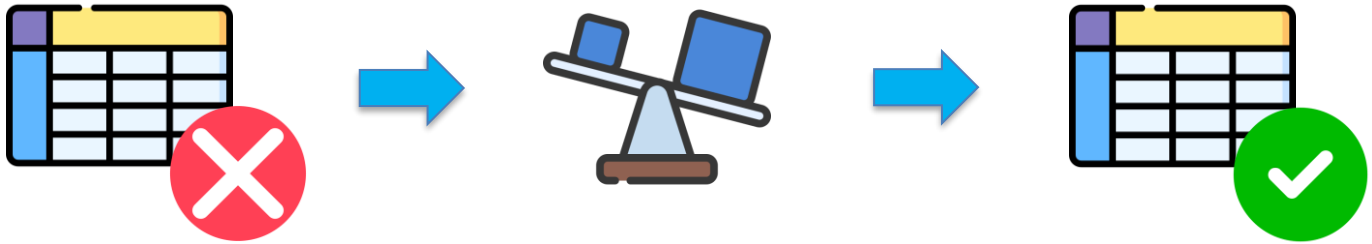


We handle that with **min-max scaling**: We apply it only to the continuous-style columns. We do NOT scale one-hot dummies or binary 0/1 flags. We build a parallel dataset **X_ready_minmax** - min-max scaled version of the continuous columns (with _mm suffix).

Target imbalance

Our target has_defaulted is heavily **imbalanced**: about ~7% defaults (1) vs ~93% non-defaults (0). So a dumb model could predict "0" for everyone and still look "accurate." That's not acceptable.

In modelling we will handle that in one or more of these ways:



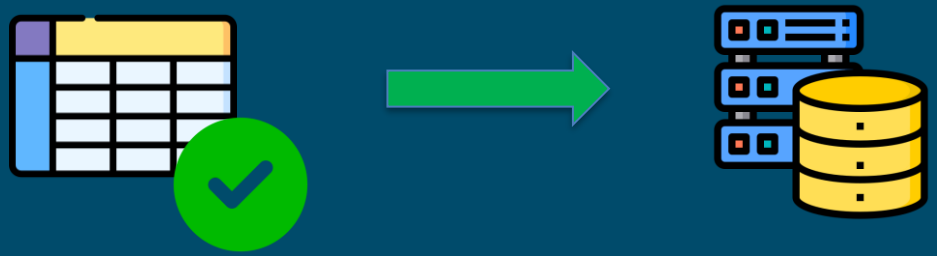
class weights (penalise missing a default more than missing a non-default),

oversampling defaulters (replicate / synthesize more rows with has_defaulted = 1 in the training split),

undersampling non-defaulters (downsample some of the 0s in training),

and using proper metrics like **recall on the default class, precision, ROC AUC, PR AUC** — not just plain accuracy.

We do not try to "balance" the dataset here by deleting good rows. We keep everyone in the ABT and handle imbalance when we train.



Thank you for your attention!
Please feel free to ask any questions

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN